

Psychometric Platform — UML Architecture & REST Endpoint Reference

An Exhaustive Technical Specification of Backend Entities, Services, Security Controllers, and Request Lifecycles

Package: `com.psychometric.platform`

Framework: Spring Boot 3.4+ / Java 21+

Audience: Backend Engineers, Architects & API Consumers

TABLE OF CONTENTS

1. Codebase Inventory & Module Overview
2. UML Class Architecture & Domain Diagrams
3. Authentication & OTP API Reference
4. Admin Management API Reference
5. Candidate Management API Reference
6. Architectural Observations & Code Quality Findings

1 Module Overview & Codebase Inventory

The backend is structured into modular feature packages under `com.psychometric.platform`, separating core security, identity, administrative management, and caching infrastructure.

1. Authentication & Identity Module (`features.auth`)

Orchestrates candidate and administrator authentication via stateless HMAC-SHA256 JWTs. Implements Redis-backed 6-digit OTP verification for self-service registration and password recovery with 5-minute TTL expirations.

2. User & Candidate Management Module (`features.user`)

Manages the `User` entity lifecycle with dual administration layers: strict `ROLE_SUPER_ADMIN` governance for administrator provisioning, and `ROLE_ADMIN` access for candidate directory management and soft-deactivation.

3. Common Security & Exception Layer (`common.*`)

Provides Spring Security configuration, Bearer token extraction filters, BCrypt hashing, XSS sanitization (`HtmlSanitizer`), and centralized error handling via `GlobalExceptionHandler`.

4. Infrastructure Layer (`infrastructure.*`)

Houses in-memory Redis caching (`RedisTemplate<String, String>`) and an asynchronous email notification engine (`EmailService`) powered by Thymeleaf HTML templates.

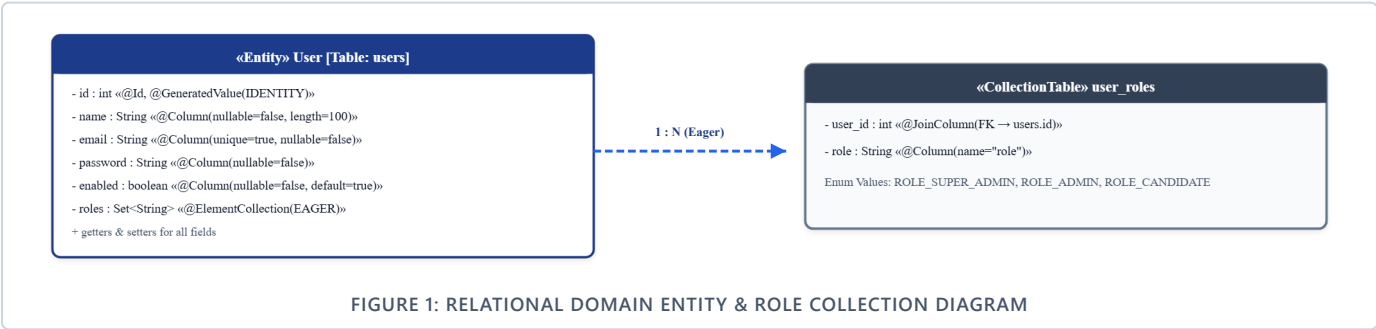
Complete File & Component Inventory (35 Source Files)

Layer / Package	Classes / Interfaces	Core Responsibility
Controllers	<code>AuthController</code> <code>AdminManagementController</code> <code>CandidateManagementController</code>	REST API endpoints for authentication, Super Admin CRUD, and Candidate administration.
Services	<code>UserService</code> <code>OtpService</code> <code>CustomUserDetailsService</code> <code>EmailService</code>	Business logic, Redis OTP caching, <code>UserDetails</code> synthesis, and async email dispatch.
Repositories & Entities	<code>UserRepository</code> <code>User</code>	JPA persistence layer mapping <code>users</code> and <code>user_roles</code> tables.
DTOs (Requests)	<code>LoginRequest</code> , <code>SignupRequest</code> , <code>OtpVerificationRequest</code> , <code>ResendOtpRequest</code> , <code>ForgotPasswordRequest</code> , <code>ResetPasswordRequest</code> , <code>ResetPasswordVerificationRequest</code> , <code>PendingSignup</code> , <code>PendingPasswordReset</code> , <code>AdminCreateRequest</code> , <code>CandidateUpdateRequest</code>	Validated input payloads with Bean Validation annotations (<code>@NotBlank</code> , <code>@Email</code> , <code>@Size</code> , <code>@Pattern</code>).
DTOs (Responses)	<code>JwtAuthenticationResponse</code> , <code>CurrentUserResponse</code> , <code>AdminResponse</code> , <code>CandidateResponse</code>	Structured API responses encapsulating tokens and user representations.
Security & Common	<code>SecurityConfig</code> , <code>JwtTokenProvider</code> , <code>JwtAuthenticationFilter</code> , <code>HtmlSanitizer</code> , <code>GlobalExceptionHandler</code> , <code>ErrorResponse</code> , Custom Exceptions (3)	JWT creation/validation, filter chain, XSS sanitization, and global REST exception advice.

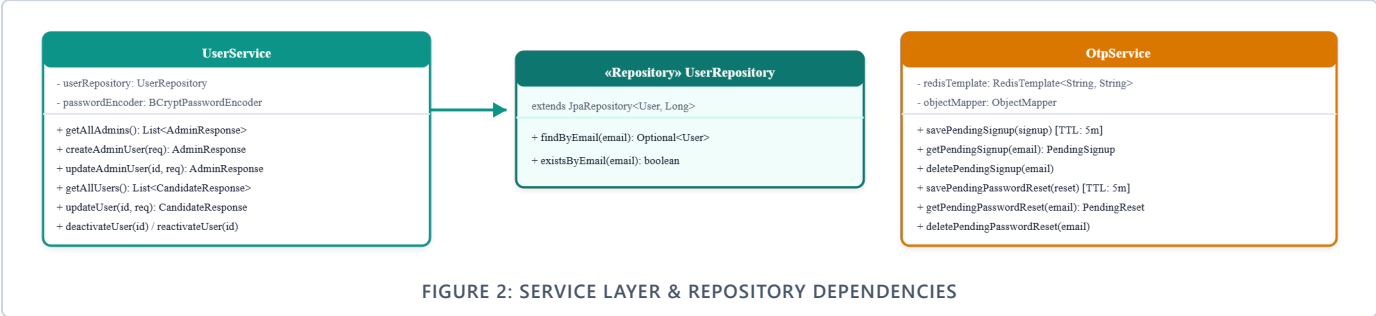
2 UML Class Diagrams & Architecture Models

The diagrams below illustrate the exact class structures, field definitions, dependency injection graphs, and controller-to-service execution flows extracted directly from the compiled codebase.

A. Entity & Domain Model Diagram



B. Service Layer Dependency & Repository Mapping



3 Controller-to-Service Mapping & Auth API

The REST Controllers map client HTTP requests directly to service and security components. Below is the controller execution graph followed by the exhaustive API endpoint specification for `AuthController`.



Authentication Endpoints (`AuthController` @ `/api/auth`)

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
POST /login	Authenticate user with credentials & issue JWT	Public	LoginRequest : - email : String (req, @Email) - password : String (req, ≥6)	200 OK JwtAuthenticationResponse { token, tokenType }	Authenticates via AuthenticationManager ; validates active user state; issues 7-day HMAC-SHA256 JWT.
POST /request-otp	Initiate self-registration and send email OTP	Public	SignupRequest : - name : String (req, 2–100, no HTML) - email : String (req, @Email) - password : String (req, ≥6)	200 OK : "Verification code sent." 409 CONFLICT if email exists	Generates 6-digit cryptographic OTP; hashes password; saves PendingSignup in Redis with 5-min TTL; dispatches email.
POST /verify-otp	Verify OTP, persist user, and auto-login	Public	OtpVerificationRequest : - email : String (req, @Email) - code : String (req, length=6)	200 OK : JwtAuthenticationResponse 401 UNAUTHORIZED if expired/invalid	Retrieves PendingSignup from Redis; verifies OTP; calls UserService.createUser (assigns ROLE_CANDIDATE); sends welcome email; purges Redis.
POST /resend-otp	Regenerate and resend signup OTP	Public	ResendOtpRequest : email : String (req, @Email)	200 OK : "OTP resent successfully." 404 NOT_FOUND if expired	Regenerates 6-digit OTP in existing Redis signup record; resets 5-minute TTL; sends updated email.
GET /me	Get authenticated user profile and roles	Bearer Auth	None (Reads Authentication from SecurityContext)	200 OK CurrentUserResponse { id, name, email, roles }	Extracts email from JWT subject; queries UserRepository ; returns active role permissions.
POST /forgot-password	Initiate password reset and dispatch reset OTP	Public	ForgotPasswordRequest : email : String (req, @Email)	200 OK : "Verification code sent." 404 NOT_FOUND if email missing	Verifies email existence; stores PendingPasswordReset in Redis with 5-min TTL; sends password reset email.
POST /verify-reset-code	Validate password reset OTP code	Public	ResetPasswordVerificationRequest : - email : String (req, @Email) - code : String (req, length=6)	200 OK : "Verification successful." 401 UNAUTHORIZED if invalid	Checks code against active Redis reset record without consuming the record yet.
POST /resend-reset-code	Resend password reset OTP code	Public	ForgotPasswordRequest : email : String (req, @Email)	200 OK : "Verification code resent." 404 NOT_FOUND if expired	Generates fresh OTP; updates Redis reset session; resets 5-min TTL; resends email.
POST /reset-password	Finalize password update with new credentials	Public	ResetPasswordRequest : - email : String (req, @Email) - password : String (req, ≥6)	200 OK : "Password reset successfully." 401 UNAUTHORIZED if expired	Verifies active reset session in Redis; encodes new password via BCrypt; updates User in DB; purges Redis reset key.

4 Admin & Candidate Management API Reference

Administrative operations are partitioned into two strict role-governed controllers to enforce separation of duties.

A. Admin Management Endpoints (AdminManagementController @ /api/admin/management)

Access Control: Enforces class-level @PreAuthorize("hasAuthority('ROLE_SUPER_ADMIN')"). Standard administrators cannot manage or elevate other admin accounts.

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
GET /	List all system administrators	ROLE_SUPER_ADMIN	None	200 OK List<AdminResponse> [{ id, name, email, roles, enabled }]	Filters users table for accounts possessing ROLE_ADMIN or ROLE_SUPER_ADMIN.
POST /	Provision a new administrator	ROLE_SUPER_ADMIN	AdminCreateRequest : • name : String (req, 2–100) • email : String (req, @Email) • password : String (req, ≥6) • roles : Set<String> (@NotEmpty)	201 CREATED AdminResponse 400 BAD_REQUEST on duplicate email	Sanitizes name via HtmlSanitizer ; encodes password with BCrypt; sets enabled=true ; saves with assigned roles.
PUT /{id}	Update administrator profile & roles	ROLE_SUPER_ADMIN	• id : int (PathVariable) • AdminCreateRequest (RequestBody)	200 OK AdminResponse 400 BAD_REQUEST on failure	Updates name, roles; updates password if non-empty; saves to database.
PATCH /{id}/reactivate	Reactivate a deactivated admin	ROLE_SUPER_ADMIN	• id : Long (PathVariable)	204 NO_CONTENT	Sets enabled = true on target user account.
DELETE /{id}	Soft-deactivate an admin account	ROLE_SUPER_ADMIN	• id : Long (PathVariable)	204 NO_CONTENT	Sets enabled = false . Prevents account login while preserving historical audit logs.

B. Candidate Management Endpoints (CandidateManagementController @ /api/candidate/management)

Access Control: Enforces class-level @PreAuthorize("hasAnyAuthority('ROLE_SUPER_ADMIN', 'ROLE_ADMIN')").

Method & Path	Purpose	Auth / Role	Request Body / Params	Response Shape	Validation & Business Rules
GET /	List all candidate test-takers	ROLE_ADMIN	None	200 OK List<CandidateResponse> [{ id, name, email, enabled }]	Filters users table for accounts possessing ROLE_CANDIDATE.
PUT /{id}	Update candidate name/email/password	ROLE_ADMIN	• id : Long (PathVariable) • CandidateUpdateRequest (RequestBody)	200 OK CandidateResponse	Sanitizes name; updates email; updates password if non-blank; saves changes.
PATCH /{id}/reactivate	Reactivate candidate assessment access	ROLE_ADMIN	• id : Long (PathVariable)	204 NO_CONTENT	Sets enabled = true . Re-enables assessment participation and portal login.
DELETE /{id}	Soft-deactivate a candidate account	ROLE_ADMIN	• id : Long (PathVariable)	204 NO_CONTENT	Sets enabled = false . Locks candidate access without deleting prior assessment records.

5 Architectural Observations & Quality Audit

A non-intrusive static code analysis of the existing codebase identified the following architectural, type consistency, and error-handling characteristics. As per the reference constraints, these are documented below for review:

1. Primary Key Type Inconsistency (`int` vs. `Long`)

In `User.java`, the primary key `id` is declared as primitive `int`. However, `UserRepository` extends `JpaRepository<User, Long>`, and several service methods (e.g. `deactivateUser(Long id)`) and controller path variables use `Long` while others use `int` (e.g. `AdminManagementController.updateAdmin(@PathVariable int id)`).
Impact: Functionally works due to autoboxing and `Long.valueOf(id)` casting, but standardizing on `Long` across entities, DTOs, and controllers is recommended during domain expansion.

2. Exception Handling Consistency Across Controllers

In `AdminManagementController`, `addAdmin` and `updateAdmin` methods wrap execution in manual `try-catch` blocks returning `ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage())`. Conversely, `CandidateManagementController` and other endpoints allow exceptions to bubble up to `GlobalExceptionHandler`.
Impact: Manual try-catch blocks produce plaintext error strings rather than the standardized JSON structure defined in `ErrorResponse.java`.

3. Generic RuntimeException Usage in UserService

In `UserService.java`, entity lookup failures throw generic `new RuntimeException("User not found")` or `new RuntimeException("Admin account not found")` instead of the custom `ResourceNotFoundException`.
Impact: `GlobalExceptionHandler` catches these as general `500 INTERNAL_SERVER_ERROR` instead of returning a clean `404 NOT_FOUND` status.

4. CORS Declaration Granularity

`AuthController` contains a class-level `@CrossOrigin(origins = "*")` annotation, whereas `AdminManagementController` and `CandidateManagementController` rely on the centralized `CorsConfigurationSource` bean in `SecurityConfig.java`.
Impact: Centralized CORS in `SecurityConfig` is the authoritative policy; the wildcard annotation on `AuthController` is redundant.

Summary for Next Phase (Domain Implementation)

The authentication and user management foundation is robust, secure, and compiling with 100% test success. These minor architectural observations can be naturally aligned when constructing the subsequent Psychometric Item Bank, Testing Session Engine, and Scoring Pipeline.